

VisionGuard

面向视障与老年群体的胸挂式智能设备系统。**硬件本地安全 + 云端数据增强 + APP 远程监护**，三端协作。

- Gitee 仓库：

gitee.com/taylorcheng/gitee/vision-guard
- 生产服务器：

<http://47.94.146.53:3000/>
- 本地开发：

<http://localhost:3000/>

给队友的快速指引

你想做什么	去哪里
安装 APP 测试	下载 apk/VisionGuard-v1.4-cloud.apk （连本地）或 cloud-deploy/android/apk/VisionGuard-v1.4-cloud.apk （连云服务器）
看后端接口	读 docs/业务流程与后端设计.md
看 UI 设计规范	读 docs/Android-UI设计文档.md
烧录硬件	固件在 hardware/esp32/esp32sense.ino ，对接指南在 docs/硬件对接文档.md
部署到云服务器	读 docs/部署指南.md ，部署包在 cloud-deploy/
看改了啥	读 docs/变更记录.md

双版本说明

项目存在两份 Android 源码，**代码基本一致，仅 BASE_URL 不同**：

版本	源码位置	BASE_URL	APK 位置
本地开发版	app/ （项目根目录）	http://127.0.0.1:3000/	apk/VisionGuard-v1.4-cloud.apk
云版	cloud-deploy/android/	http://47.94.146.53:3000/	cloud-deploy/android/apk/VisionGuard-v1.4-cloud.apk

修改 Android 代码时需同步两份源码，确保 cloud-deploy/android/.../RetrofitClient.kt 的 BASE_URL 不被覆盖。



目录

章节	内容	适合谁
一、产品概述	使用场景 + 硬件组成	所有人
二、系统架构	三端架构图 + 数据流	所有人
三、核心业务流程	设备认证 / 摔倒告警 / 药品识别	所有人
四、入门导航	不同角色从哪开始	新人必读
五、技术栈	各端技术选型	开发者
六、认证体系	设备认证 vs 用户认证	开发者
七、数据库概览	17 张表分组说明	Android / 后端
八、项目结构	完整目录树	所有人
九、文档索引	11 份文档的受众/内容	所有人

章节	内容	适合谁
十、后端接口总览	81 路由完整清单	Android / 硬件 / 后端
十一、本地开发	环境搭建 + 启动	后端
十二、测试指南	后端自测 + 硬件联调测试	后端 + 硬件
十三、云服务器部署	Docker 一键部署 + 参数对照	后端
十四、项目状态	已完成 / 待做	所有人
十五、团队协作	接口约定 + 开发流程 + 沟通清单	所有人

一、产品概述

一台挂在胸前的小型设备（ESP32 + K210），**本地实时检测摔倒、障碍物、环境声音**并语音播报——断网也能用。联网时自动上传数据到云端，监护人在 Android APP 上查看位置、告警、健康数据。

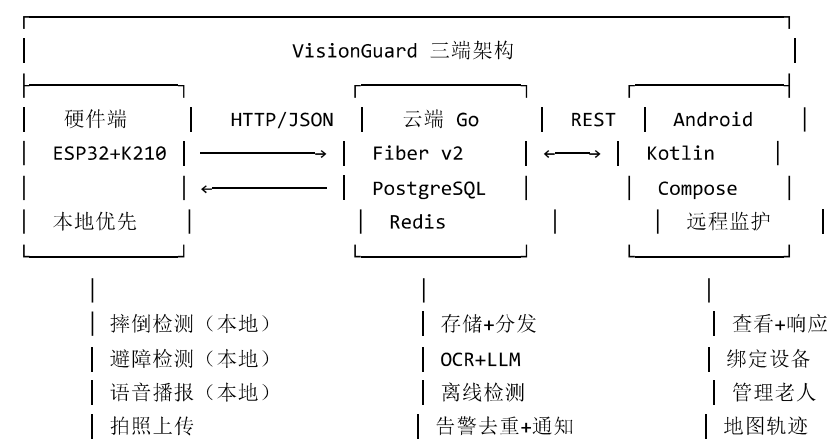
使用场景

场景	硬件做什么	云端做什么	APP 做什么
摔倒	MPU6050 检测姿态异常 → 本地声光告警	存储告警记录 + 推通知给监护人	弹出告警 + 显示位置
避障	K210 AI 识别障碍物方位 → 语音"左/中/右"	记录障碍告警	无
识药	按键拍照 → K210 拍药品 → 上传	OCR 识别药品名 + LLM 用药建议	显示结果 + 语音播报
定位	GPS 实时定位	存储轨迹 + 电子围栏检测	地图展示 + 越界告警
健康	心率/血压/血氧传感器	存储 + 趋势分析	图表展示

硬件组成

模块	型号	作用
主控+联网	ESP32	WiFi、传感器驱动、语音播放、云端通信
AI 视觉	K210	障碍物检测、拍照、JPEG 压缩
姿态检测	MPU6050	陀螺仪+加速度计，判断摔倒
激光测距	VL53L5CX	多区域障碍物距离检测
定位	GPS/北斗	实时经纬度，所有数据带位置
语音	I2S + 喇叭	本地汉字语音库逐字朗读
存储	SPIFFS	1000 字中文 WAV 语音库

二、系统架构



数据流方向：

- 上行（硬件 → 云 → APP）：心跳、位置、告警、健康数据、OCR 图片
- 下行（APP → 云 → 硬件）：绑定确认、OCR 文本结果
- 本地闭环：摔倒/避障检测 → 本地语音告警（不经过云端）

两个环境：

	本地开发	云服务器
地址	http://localhost:3000	http://47.94.146.53:3000
数据库	本地直装 PostgreSQL + Redis	Docker 容器（自动启动）
用途	写代码 + 本地测试 + 硬件联调	正式上线，对外开放
部署方式	go run cmd/server/main.go	./deploy.sh（Docker）

三、核心业务流程

1. 设备安全认证（4 步）

激活(一次) → 注册(每次上电) → Challenge-Response → 获取 JWT → 正常通信

↓

XOR(deviceSecret+nonce+timestamp, 0x4B)

一次性挑战码, Redis TTL 5 分钟

2. 摔倒告警全链路

MPU6050 姿态异常

- ESP32 本地声光告警 + 语音"摔倒报警"
- POST /api/v1/alert (alertType=fall, 带 GPS)
- 云端去重检查(120s 窗口)
- 查绑定关系 → 找到监护人
- 创建 notification 记录
- APP 轮询 /api/v1/notifications → 弹出告警 + 地图显示位置

3. 药品识别流程

按键 A → K210 拍照 → JPEG 压缩 → ESP32 上传
→ POST /api/v1/ocr/image (图片元信息)
→ POST /api/v1/ocr/recognize (创建识别任务)
→ 异步 OCR (mock 3s) → LLM 用药建议 (mock 5s)
→ APP 轮询 /api/v1/ocr/poll/:taskId → 获取结果
→ 云端返回纯文本 → ESP32 逐字朗读本地 WAV 语音库

四、入门导航

不同角色从哪开始

角色	第一步	第二步	日常参考
硬件开发	硬件对接文档 — 接口+认证+curl	硬件架构说明 — K210/ESP32 串口协议	硬件对接文档第八章
Android 开发	业务流程与后端设计 — 全部接口 +DB	Android 开发指引 — 工程结构	业务流程文档第 2 章接口清单
后端开发	业务设计原始版 — 原始规格	业务流程与后端设计 — 实现文档	backend/internal/ 源码
文档同学	↓ 下方文档撰写指南 — 项目全貌速览	业务流程与后端设计	本文件 + 所有 docs/ 文档
学长/负责人	本文件 (README)	业务流程与后端设计	代码审查清单

如果你是文档同学

负责撰写项目文档（产品说明书、用户手册、答辩 PPT、论文等），需要快速理解项目全貌、技术架构和业务流程。

建议阅读顺序（约 30 分钟建立全局认知）：

步骤	读什么	花多久	收获
1	本文件 §一～§三（产品概述+架构+核心流程）	10 min	项目是什么、三端怎么协作
2	本文件 §五（技术栈）	5 min	各端用了什么技术
3	业务流程与后端设计 §一～§三	10 min	完整业务模块 + 核心数据流
4	本文件 §十四（项目状态）	5 min	哪些做完了、哪些待做

项目三句话（用于答辩开场/论文摘要）：

- 1. VisionGuard 是一款**胸挂式智能设备**，ESP32+K210 在本地实时检测摔倒、障碍物并语音播报，**断网也能用**
- 2. 联网时数据上传 Go 云端（PostgreSQL+Redis），监护人通过 Android APP 远程查看**位置、告警、健康数据**
- 3. 设备→云端→APP 三端通过 HTTP REST API 通信，设备用 XOR challenge-response 认证，用户用 JWT

关键数字（答辩素材）：

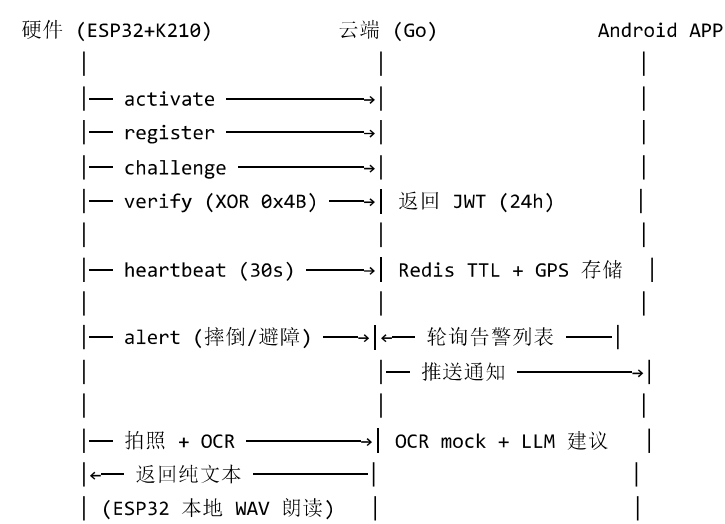
- 后端 77 条 REST 路由、16 张数据表
- 7 种告警类型（摔倒/避障/SOS/心率/低电量/离线/围栏）
- Android 18 个页面、5 个 Tab、全局统一设计规范

- 设备认证：XOR 0x4B challenge-response + JWT 24h

技术栈速览：

端	语言	框架/关键库	数据库
硬件	C++ (ESP32) + Python (K210)	Arduino, MaixPy	—
云端	Go 1.23	Fiber v2, GORM, JWT	PostgreSQL 16 + Redis 7
Android	Kotlin	Jetpack Compose, Retrofit, ZXing, 高德地图	—

三端通信流程图（用于答辩 PPT）：



文档撰写常用参考：

- 完整接口清单 → docs/业务流程与后端设计.md §二
- 数据库表结构 → docs/业务流程与后端设计.md §三 或 本文件 §七
- 产品功能需求 → docs/产品需求说明书.md
- Android 页面/设计规范 → docs/Android-UI设计文档.md
- 硬件规格 → hardware/README.md

五、技术栈

后端

层	技术	说明
语言	Go 1.23+	
HTTP 框架	Fiber v2	类 Express，性能优先
ORM	GORM	AutoMigrate 自动建表
数据库	PostgreSQL 16	17 张业务表
缓存	Redis 7	Challenge 暂存、心跳状态、位置缓存
认证	JWT HS256 + bcrypt + XOR 0x4B	三重机制

Android

层	技术	说明
语言	Kotlin	JVM 17 目标
UI 框架	Jetpack Compose + Material 3	声明式 UI，全局设计规范
导航	Compose Navigation	5 Tab + 子页面路由
HTTP	Retrofit 2 + OkHttp	Token 自动续期（401 Authenticator）
序列化	Gson	配合 Retrofit Converter
异步	Kotlin Coroutines + Flow	Dispatchers.IO 网络；StateFlow 状态管理
图片加载	Coil (Compose)	异步加载、缓存、占位符
地图 SDK	高德 3D 地图 10.0.600	TextureMapView + Compose 互操作
扫码	ZXing Android Embedded 4.3	竖屏扫描 + 蓝色主题
ML Kit	Google ML Kit Text Recognition	备用 OCR 能力
推送	Firebase Cloud Messaging	通知推送
AI 推理	ONNX Runtime Android	本地模型推理（预留）

硬件

端	技术
硬件	ESP32 (Arduino), K210 (MicroPython), MPU6050, VL53L5CX

六、认证体系

系统认证分两条独立链路：

设备认证（硬件 → 云端）

- XOR 0x4B Challenge-Response → 设备 JWT (24h 有效)
- 密钥 0x4B，逐字节异或，hex 编码
 - Challenge 一次性 + Redis TTL 5 分钟
 - JWT claims 含 type="device"
 - 中间件 DeviceAuth 校验 type=="device"

密钥交换流程：

1. 硬件 POST /device/challenge → 云端返回 challengeId, nonce, timestamp
2. 硬件 本地 XOR(deviceSecret+nonce+timestamp, 0x4B) → 得到 sign
3. 硬件 POST /device/verify {sign} → 云端比对，签发 JWT
4. 硬件 后续所有请求带 Authorization: Bearer <jwt>

用户认证（APP → 云端）

- 用户名+密码 → bcrypt 比对 → 用户 JWT (1h) + RefreshToken (30d)
- 密码 ≥8 位，bcrypt DefaultCost 哈希
 - 8 次登录失败 → 账户锁定 30 分钟
 - JWT claims 含 type="user"
 - 中间件 UserAuth 校验 type=="user"
 - Token 刷新：先建新再删旧（防崩溃丢失）
 - 登出：删除 RefreshToken

七、数据库概览（16 张表）

用户与认证

表	关键字段	敏感处理
users	username, password_hash, email, phone, status	password_hash json:"-"
refresh_tokens	token_hash, user_id, expires_at	token_hash json:"-"
auth_logs	device_id, log_type, message	—

老人与监护

表	说明
elders	老人档案（姓名/性别/血型/过敏/病史/状态）
emergency_contacts	紧急联系人（FK→elders）
guardianships	监护关系（primary/normal，多对多）
invitations	监护邀请（48h 过期）
transfers	主监护人转让（双因子确认）

设备

表	说明
devices	设备主档案（device_code 敏感，状态/电池/RSSI/位置）
bindings	设备-老人绑定（pending_device_confirm/bound/unbound）

业务数据

表	说明
alerts	告警（7 类型：fall/obstacle/sos/heart_rate/low_battery/offline/geofence）
notifications	推送通知（P0-P3 优先级，app/sms/voice_call 渠道）
ocr_records	OCR 识别（药品匹配+LLM 建议+用户反馈）
locations	定位轨迹
geofences	电子围栏（圆形/多边形）

表	说明
health_data	健康数据（心率/血压/步数/血氧）

完整表结构（含字段类型、索引、关联）见 [业务流程与后端设计](#) 第 5 章。

八、项目结构

vision-hub/	# ★ Gitee: gitee.com/taylorchengitee/vision-guard
└─ apk/	# 📱 发布 APK（直接安装）
└─ VisionGuard-v1.4.1-local.apk	# ★ 本地版（127.0.0.1:3000）
└─ app/	# 🤖 Android 监护端（Kotlin + Compose, 18 页面）
└─ src/main/java/.../ui/screens/	
└─ backend/	# ☁️ Go 云端服务（Fiber + GORM + PostgreSQL + Redis）
└─ cmd/server/main.go	# 入口（17 表 AutoMigrate + 81 路由）
└─ internal/handler/	# 10 个 HTTP 处理器
└─ internal/service/	# 8 个业务逻辑层
└─ internal/model/	# 17 个 GORM 模型
└─ internal/middleware/	# JWT 认证中间件
└─ test_all_full.go	# 76 步全路由测试
└─ hardware/	# 🛠️ 硬件固件
└─ esp32/esp32sense.ino	# ★ ESP32 固件（WiFi: wuiPhone 16, 指向云服务器）
└─ k210/	# K210 AI 视觉（main.py + detect.kmodel）
└─ cloud-deploy/	# 🚀 云服务器部署包（后端源码 + Docker + 三端完整交付）
└─ cmd/server/main.go	# 入口（81 路由, 17 表 AutoMigrate）
└─ internal/	# Go 后端源码（与 backend/ 同步）
└─ Dockerfile + docker-compose.prod.yml + .env.example	
└─ android/	
└─ apk/VisionGuard-v1.4-cloud.apk	# ★ 云版（47.94.146.53:3000）
└─ hardware/	# 硬件固件副本
└─ Dockerfile	# Docker 生产部署
└─ docs/	# 📄 文档（13 份）
└─ 部署指南.md	# ★ 生产部署步骤
└─ 硬件对接文档.md	# ESP32 对接指南
└─ 业务流程与后端设计.md	# 81 路由 + 17 表 + 架构
└─ 变更记录.md	# 版本变更历史
└─ ...	
└─ CLAUDE.md	# AI 开发指引
└─ README.md	# 本文件

九、文档索引

文件	给谁	内容
硬件对接文档	硬件团队	4步认证流程、XOR 0x4B C++ 完整代码、新旧接口对照、curl 测试脚本、本地测试指南、常见问题排查

文件	给谁	内容
业务流程与后端设计	学长 / Android	81 路由完整清单（含请求/响应）、6 大核心业务流、17 表结构、认证体系详解、安全设计、测试指南、Docker 部署
产品需求说明书	全团队	产品定位、功能需求、硬件规格、交互流程
安卓说明文档	全团队	三端业务边界定义、P0/P1 功能模块列表、串口通讯协议
Android-UI设计文档	Android	UI11.DOCX 设计规范：4 Tab + 色值圆角字体 + 页面结构 + 后端接口对照
api.md	硬件团队（存档）	旧版 6 接口文档（端口 8888），供参考
业务设计原始版	学长（存档）	10 大业务模块原始规格，当前实现的依据
代码审查清单	后端	31 文件对账、77 路由核对、61 安全检查点
数据流模拟	全团队	端到端数据流场景模拟
开发日志	后端	按日记录：环境搭建、代码实现、Bug 修复、网络联调
变更记录	全团队	版本变更摘要

十、后端接口总览（77 路由，10 模块）

一、认证服务（8 路由）

方法	路径	认证	说明
POST	/api/v1/device/challenge	无	请求挑战码（XOR 0x4B，Redis TTL 5min）
POST	/api/v1/device/verify	无	提交签名验证，返回设备 JWT（24h）
POST	/api/v1/device/register	无	设备首次接入注册（记录 IP）
POST	/api/v1/device/info	无	记录设备型号/固件版本
POST	/api/v1/device/log	无	记录设备认证事件日志
POST	/api/v1/auth/register	无	用户注册（bcrypt，密码 ≥8 位）
POST	/api/v1/auth/login	无	登录（JWT 1h，8 次失败锁账户）
POST	/api/v1/auth/refresh	无	刷新 Token（翻转机制）
POST	/api/v1/auth/logout	无	登出（删除 refresh_token）

二、老人档案与监护关系（15 路由，UserAuth）

方法	路径	说明
POST	/api/v1/elder	创建档案（自动设为主监护人）
GET	/api/v1/elder/:elderId	查询档案详情
PUT	/api/v1/elder/:elderId	更新（主监护人专权）
DELETE	/api/v1/elder/:elderId	删除（事务解绑所有设备 + 级联删子表）
POST	/api/v1/elder/:elderId/archive	归档封存（自动解绑设备）

方法	路径	说明
POST	/api/v1/elder/:elderId/guardian/invite	邀请协作监护人（48h 过期）
POST	/api/v1/elder/:elderId/guardian/accept	接受邀请（身份验证）
DELETE	/api/v1/elder/:elderId/guardian/:userId	移除监护人（主可移他人，普通可移自己）
POST	/api/v1/elder/:elderId/primary/transfer	发起主监护人转让
POST	/api/v1/elder/:elderId/primary/confirm	确认转让（角色互换）
POST	/api/v1/elder/:elderId/emergency-contact	添加紧急联系人
DELETE	/api/v1/elder/:elderId/emergency-contact/:contactId	删除紧急联系人
POST	/api/v1/elder/:elderId/bind	绑定设备到老人
GET	/api/v1/elders	我监护的老人列表
GET	/api/v1/dashboard	监护人仪表盘（含24h告警计数）

三、设备接入（8 路由）

方法	路径	认证	说明
POST	/api/v1/device/activate	无	激活（分配 deviceSecret）
POST	/api/v1/device/auth	无	设备认证获取 Token
PUT	/api/v1/device/:deviceId	DeviceAuth	更新设备信息
POST	/api/v1/device/:deviceId/toggle	DeviceAuth	禁用/启用
GET	/api/v1/device/:deviceId/firmware	DeviceAuth	固件版本查询
POST	/api/v1/device/:deviceId/data	DeviceAuth	数据上报

四、心跳与在线状态（5 路由）

方法	路径	认证	说明
POST	/api/v1/device/heartbeat	DeviceAuth	心跳（30s 间隔，Redis TTL 180s）
GET	/api/v1/device/status/:deviceId	DeviceAuth	在线状态（Redis 优先）
GET	/api/v1/device/:deviceId/last-online	DeviceAuth	最后在线时间
POST	/api/v1/devices/batch-status	UserAuth	批量设备状态
—	离线检测 goroutine	内部	每 10s 扫描 → 90s 无心跳 → 标记 offline + 通知

五、设备绑定（7 路由，UserAuth）

方法	路径	说明
GET	/api/v1/device/:deviceId/search	搜索设备（判断可绑定状态）
POST	/api/v1/binding/initiate	发起绑定（监护人+老人+设备状态校验）
POST	/api/v1/binding/confirm	设备端确认（5min 超时）
POST	/api/v1/binding/check	唯一绑定约束校验
POST	/api/v1/binding/unbind	解绑（主监护人专权）

方法	路径	说明
POST	/api/v1/binding/rebind	换绑（双重权限校验）
GET	/api/v1/device/:deviceId/binding	查询绑定关系

六、设备数据接收（2 路由）

方法	路径	认证	说明
POST	/api/v1/data/health	无	健康数据接收（心率/血压/步数/血氧）
GET	/api/v1/data/health	UserAuth	历史健康数据查询（分页）

七、告警管理（8 路由）

方法	路径	认证	说明
GET	/api/v1/alert/types	无	7 种告警类型列表
POST	/api/v1/alert	无	上报告警（去重窗口：fall 120s 等）
GET	/api/v1/alerts	UserAuth	告警历史（分页+过滤）
GET	/api/v1/alert/statistics	UserAuth	统计（按类型/等级/状态，按天/周/月）
GET	/api/v1/alert/level-config	UserAuth	等级配置
GET	/api/v1/alert/:alertId	UserAuth	告警详情（含老人/设备+时间线）
PUT	/api/v1/alert/:alertId/status	UserAuth	更新状态（confirm/resolve/close）
POST	/api/v1/alert/:alertId/resolve	UserAuth	解决告警

八、定位（7 路由，UserAuth）

方法	路径	说明
GET	/api/v1/location/latest	最新位置（Redis→DB）
GET	/api/v1/location/trajectory	历史轨迹
GET	/api/v1/location/alert-markers	告警地图标记
GET	/api/v1/device/:deviceId/running	设备运行数据
POST	/api/v1/geofence	创建围栏（圆形/多边形）
GET	/api/v1/geofences	围栏列表
DELETE	/api/v1/geofence/:fenceId	删除围栏

九、OCR（7 路由，UserAuth）

方法	路径	说明
POST	/api/v1/ocr/image	图片上传记录
POST	/api/v1/ocr/recognize	创建 OCR 任务（异步 mock 3s）
GET	/api/v1/ocr/result/:taskId	OCR 结果
GET	/api/v1/ocr/poll/:taskId	任务状态轮询

方法	路径	说明
POST	/api/v1/ocr/suggestion	LLM 用药建议（异步 mock 5s）
POST	/api/v1/ocr/feedback	识别反馈
GET	/api/v1/ocr/records	历史记录

十、通知（8 路由，UserAuth）

方法	路径	说明
GET	/api/v1/notifications	消息列表（分页+过滤）
PUT	/api/v1/notifications/read	标记已读（归属校验）
PUT	/api/v1/notifications/read-all	全部已读
GET	/api/v1/notification/push-rules	推送规则
POST	/api/v1/notification/push-targets	推送目标（等级→渠道）
POST	/api/v1/notification/push	发送推送（stub）
GET	/api/v1/notification/status/:messageId	推送状态
GET	/api/v1/notification/priority-config	优先级配置（P0-P3）

健康检查（1 路由）

方法	路径	认证	说明
GET	/api/v1/healthz	无	{"status": "ok"}

十一、本地开发

- 地址：http://localhost:3000
- 前提：PostgreSQL 16 + Redis 7 本地安装并运行

```
# 1. 配置环境
cd backend
cp .env.example .env
# 本地开发 .env 关键配置:
#   SERVER_PORT=3000
#   DB_HOST=localhost           ← 本地直连
#   REDIS_HOST=localhost        ← 本地直连
#   DB_USER=postgres           ← 本地 PostgreSQL 用户

# 2. 安装依赖 + 编译
go mod tidy
go build ./...

# 3. 启动（自动建 16 张表）
go run cmd/server/main.go
# → VisionGuard backend starting on :3000

# 4. 验证
curl http://localhost:3000/api/v1/healthz
# → {"status":"ok"}

# 5. 运行测试（另开终端）
go run test_all.go           # 21 步核心流程
go run test_all_full.go      # 76 步全路由覆盖
bash test_e2e.sh             # 端到端模拟（硬件→后端→APP）
python3 test_ocr.py          # OCR 全链路（设备认证→上传→豆包识别→轮询）
```

Android 真机联调

手机开热点时，手机自身 APP 无法直连电脑后端（Android 热点流量走蜂窝网）。需用 ADB 反向隧道：

```
# 先装 ADB（如未安装）
# 下载 platform-tools → 解压到 C:\Users\HONOR\AppData\Local\Android\Sdk\platform-tools\

# 手机 USB 连电脑，建立反向隧道
adb reverse tcp:3000 tcp:3000

# Android 端 BASE_URL 设为（在 RetrofitClient.kt）
http://127.0.0.1:3000/
```

十二、测试指南

12.1 后端自测（不需要硬件）

前提：本地后端已启动（见 [第十一章 本地开发](#)）。

```
cd backend

# 21 步核心流程测试（覆盖关键链路：注册→登录→设备激活→认证→心跳→绑定→告警→OCR→通知）
go run test_all.go

# 76 步全路由覆盖测试（逐一验证全部 77 路由，一键跑完）
go run test_all_full.go
```

预期结果：

test_all.go: 21 PASS, 0 FAIL
test_all_full.go: 76 PASS, 0 FAIL

测试覆盖的业务链路：

test_all.go (21 步)	test_all_full.go (76 步)
用户注册登录	用户注册登录 + Token 刷新
设备激活 + 注册 + XOR 认证	设备激活 + 注册 + Challenge + Verify + info + log
创建老人 + 绑定设备	创建老人 + 查询 + 更新 + 紧急联系人 + 绑定
心跳上报	监护人邀请 + 接受 + 转让 + 确认 + 转让回来
上报告警 + 查看	设备心跳 + 状态 + 固件 + 数据上报 + 批量
OCR 图片上传 + 识别	绑定发起 + 确认 + 检查 + 关系查询 + 换绑 + 解绑
通知列表 + 已读	健康数据保存 + 查询
健康检查	告警类型 + 创建 + 统计 + 历史 + 详情 + 状态 + 解决
	最新位置 + 轨迹 + 地图标记 + 运行数据 + 围栏 CRUD
	OCR 上传 + 识别 + 轮询 + LLM 建议 + 反馈 + 历史
	通知列表 + 已读 + 全部已读 + 推送规则 + 推送
	老人归档 + 删除 + 健康检查

12.2 硬件接入规范

ESP32 必须通过以下条件连接 WiFi，否则会连接失败：

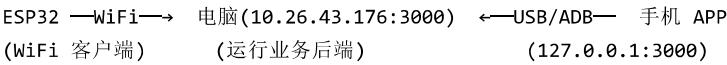
规范	要求	说明
安全协议	WPA2-Personal	ESP32 Arduino SDK 不支持 WPA3，不要使用 WPA3 或 WPA3/WPA2 混合
频段	2.4 GHz	ESP32 不支持 5GHz WiFi
SSID 命名	仅英文字母/数字	中文 SSID（如"荣耀400"）会导致 ESP32 编码匹配失败
密码	无特殊要求	WPA2 密码理论上可达 63 字符

⚠️ 手机热点默认可能是 WPA3 或混合模式，确认设置为 「WPA2-Personal」。

12.3 硬件联调测试（ESP32 + 后端 + Android）

前提：手机开热点「Honor400」（WPA2-Personal / 2.4GHz，密码 czj20070312），电脑连热点（IP 固定为 10.26.43.176），后端已启动。

网络拓扑：



第一步：启动后端

```
cd backend && go run cmd/server/main.go
```

第二步：ESP32 烧录固件

ESP32 固件（ hardware/esp32/esp32sense.ino ）已预配 WiFi：

```
const char* BASE_URL = "http://10.26.43.176:3000";
const char* ssid      = "Honor400";
const char* password  = "czj20070312";
```

第三步：手机 ADB 隧道

```
adb reverse tcp:3000 tcp:3000
```

第四步：逐步验证

步骤	操作	预期
1	ESP32 上电	串口输出 DEVICE_ONLINE
2	ESP32 心跳	每 30s 后端收到 heartbeat
3	手机 APP 登录	首页显示"设备在线"
4	模拟摔倒	APP 收到告警通知

无硬件时用 curl 模拟：

```
# 端到端全链路（硬件→后端→APP）
cd backend && bash test_e2e.sh
SN="SN_CURL_TEST_$(date +%s)"

# Step 1: 激活
RESP=$(curl -s -X POST $BASE/api/v1/device/activate \
  -H "Content-Type: application/json" \
  -d "{\"serialNo\":\"$SN\",\"model\":\"ESP32_K210\",\"mac\":\"AA:BB:CC:DD:EE:FF\",\"hwVersion\":\"1.0\",\"fwVersion\":\"1.0
DEVICE_ID=$(echo $RESP | python3 -c "import sys,json; print(json.load(sys.stdin)['data']['deviceId'])")
DEVICE_SECRET=$(echo $RESP | python3 -c "import sys,json; print(json.load(sys.stdin)['data']['deviceSecret'])")
echo "deviceId=$DEVICE_ID"

# Step 2: 注册
curl -s -X POST $BASE/api/v1/device/register \
  -H "Content-Type: application/json" \
  -d "{\"deviceId\":\"$DEVICE_ID\",\"deviceModel\":\"ESP32_K210\",\"firmwareVersion\":\"1.0.0\"}"

# Step 3: 请求挑战
RESP=$(curl -s -X POST $BASE/api/v1/device/challenge \
  -H "Content-Type: application/json" \
  -d "{\"deviceId\":\"$DEVICE_ID\"}")
CHALLENGE_ID=$(echo $RESP | python3 -c "import sys,json; print(json.load(sys.stdin)['challengeId'])")
NONCE=$(echo $RESP | python3 -c "import sys,json; print(json.load(sys.stdin)['nonce'])")
TIMESTAMP=$(echo $RESP | python3 -c "import sys,json; print(json.load(sys.stdin)['timestamp'])")

# Step 4: XOR 签名并验证
PLAINTEXT="{DEVICE_SECRET}${NONCE}${TIMESTAMP}"
SIGN=$(python3 -c "import binascii; pt='$PLAINTEXT'; r=bytes([ord(c)^0x4b for c in pt]); print(binascii.hexlify(r).decode())")
JWT=$(curl -s -X POST $BASE/api/v1/device/verify \
  -H "Content-Type: application/json" \
  -d "{\"deviceId\":\"$DEVICE_ID\",\"challengeId\":\"$CHALLENGE_ID\",\"sign\":\"$SIGN\"}" \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['jwt'])")
echo "JWT acquired: ${JWT:0:20}..."

# Step 5: 心跳
curl -s -X POST $BASE/api/v1/device/heartbeat \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $JWT" \
  -d "{\"deviceId\":\"$DEVICE_ID\",\"timestamp\":\"$(date +%s)\",\"battery\":85,\"rssi\":-55,\"location\":{\"lat\":31.2304,\"lng

# Step 6: 上报告警
curl -s -X POST $BASE/api/v1/alert \
  -H "Content-Type: application/json" \
  -d "{\"deviceId\":\"$DEVICE_ID\",\"timestamp\":\"$(date +%s)\",\"alertType\":\"fall\",\"alertLevel\":\"critical\",\"description

echo -e "\n=== 全部 6 步完成 ==="
```



12.3 常见联调问题

现象	原因	解决
Connection refused	后端未启动或端口错误	确认 go run cmd/server/main.go 正在运行，端口 3000
Connection timed out	防火墙阻挡	以管理员身份执行防火墙放行命令
Network is unreachable	不在同一网络	ESP32 和电脑连同一个 WiFi；检查电脑 IP 是否正确
challenge expired	超过 5 分钟才验证	重新请求 challenge，立即计算 sign 提交

现象	原因	解决
invalid signature	XOR 计算错误	检查 deviceSecret 正确性、明文拼接顺序
401 Unauthorized	JWT 过期（设备 24h）	重新执行 challenge → verify 获取新 JWT
device disabled	设备被后台禁用	调用 /api/v1/device/:deviceId/toggle 启用

更详细的联调指南见 [硬件对接文档](#) 第八章。

十三、云服务器部署

- 生产地址：

http://47.94.146.53:3000
- 前提：

服务器安装 Docker，代码已 push 并 pull 到服务器

```
cd cloud-deploy

# 创建 .env（从旧容器提取或从 .env.example 复制）
docker inspect visionguard-backend-1 --format '{{range .Config.Env}}{{println .}}{{end}}' \
  | grep -vE '^(PATH|=|TZ=)' > .env 2>/dev/null || cp .env.example .env

# ⚠️ 编辑 .env，云服务器必须改的参数：
#   SERVER_PORT=3000
#   DB_HOST=postgres          ← Docker 容器间用服务名通信，不是 localhost!
#   REDIS_HOST=redis          ← 同上
#   DB_USER=postgres
#   DB_PASSWORD=visionhub
#   JWT_SECRET=fabd660f3319c6532f917aa82004aa846157fd41c0b52689268c3f0fde3db99b

# 构建并启动（注意：docker compose，不是 docker-compose）
docker compose -f docker-compose.prod.yml up -d --build

# 验证
curl http://localhost:3000/api/v1/healthz
# → {"status":"ok"}
```

本地 vs 云端 .env 对照

参数	本地开发	云服务器	说明
SERVER_PORT	3000	3000	一致
DB_HOST	localhost	postgres	云端用 Docker 服务名
REDIS_HOST	localhost	redis	同上
DB_USER	postgres	visionhub	Docker 镜像默认用户名不同
DB_PASSWORD	你本地设的	.env.example 随机值	云端用强密码
JWT_SECRET	你本地设的	.env.example 随机值	云端用强密钥
OCR_SERVICE_URL	localhost:8001	真实 OCR 地址	当前 mock，对接后改
LLM_API_URL	localhost:8002	真实 LLM 地址	当前 mock，对接后改

Docker 部署结构

```
docker compose -f docker-compose.prod.yml up -d
├─ postgres:16-alpine    (内部 5432, 数据卷 pgdata)
├─ redis:7-alpine        (内部 6379)
└─ backend                (端口映射 3000:3000)
    └─ 通过容器名 postgres / redis 连接数据库
```

十四、项目状态

项	状态	备注
后端编译	✓	go build ./... 零错误
77 路由注册	✓	全部在 cmd/server/main.go 注册
76 步全路由测试	✓	test_all_full.go 全部 PASS
端到端模拟测试	✓	test_e2e.sh 14/15 PASS
Android 真机联调	✓	创建老人/修改密码/换绑 全部通过
16 张数据库表	✓	GORM AutoMigrate 自动创建
设备 XOR 认证	✓	Challenge-Response + JWT (设备 24h / 用户 1h)
代码审查	✓	4 轮审查, 61 检查点通过
Android 开发	✓	18 页面 + 全局下拉刷新 + 核心流程贯通 + 真机验证通过, v1.3.3 APK 已签名
硬件对接文档	✓	含本地测试指南 + curl 脚本 + 故障排查
Android 对接文档	✓	77 路由 + 业务流 + DB + 安全 + 部署
Docker 部署	✓	多阶段构建 + compose + 一键脚本
云上线	✓	http://47.94.146.53:3000/
OCR/LLM 服务	Mock	异步 3s/5s 延时, 待对接真实服务
推送渠道	Stub	短信/语音电话推送待对接
电子围栏/健康数据/监护人邀请等 UI	待开发	API 客户端已定义, 属于锦上添花功能
单元测试	未开始	核心 service 层待补

十五、团队协作

三端接口约定

- **协议**: HTTP/JSON, 统一前缀 /api/v1/
- **认证**: 硬件拿 DeviceAuth JWT, APP 拿 UserAuth JWT, 部分接口无认证
- **字段命名**: 全部 camelCase (JSON tag), 请求和响应一致
- **错误格式**: {"code": 400, "message": "描述"}

你需要改端口 / IP 时

文件	改什么
backend/.env	SERVER_PORT=xxxx , DB_HOST=... , REDIS_HOST=...
backend/.env.example	同上（模板）
backend/docker-compose.prod.yml	ports: - "xxxx:xxxx"
backend/deploy.sh	末尾健康检查的 URL 端口
backend/test_all.go	const base = "http://localhost:xxxx"
backend/test_all_full.go	同上
app/.../RetrofitClient.kt	const val BASE_URL
README.md	所有涉及地址的地方

沟通清单

如果改了	需要通知谁	更新什么文档
接口路径/参数	硬件 + Android	硬件对接文档 + 业务流程文档
认证流程	硬件 + Android	硬件对接文档（XOR 章节） + 业务流程文档
数据库表结构	Android	业务流程文档（DB 章节）
服务器地址/端口	所有人	README + 各端代码中的地址